Functional Design Document (FDD)

Project: Researcher Knowledge Base and Ranking System

Version: 1.0 (Current-state, till now)

Date: 2026-05-07

1. Purpose

This document defines the functional behavior of the system as implemented to date. The platform has two functional parts:

- Offline knowledge base pipeline (Stages 1-4)
- Online ranking API (`/rank`, `/health`)

The goal is to identify, enrich, and rank researchers for a user query using both semantic relevance and impact signals.

2. Business Objectives

- Build a reusable researcher knowledge base for selected topics.
- Rank researchers with explainable signals rather than citations alone.
- Support fast query-time ranking via API.
- Allow filtering by region and institution.

3. In-Scope (Current)

- Topic-driven paper discovery from OpenAlex.
- Author extraction and de-duplication across topics.
- Author, coauthor/mentee, and institution enrichment.
- Final assembly of structured researcher profiles.
- Batch upsert to MySQL during Stage 4.
- Semantic ranking API with mock mode and live mode.

4. Out-of-Scope (Current)

- Frontend UI.
- Full multi-tenant authentication/authorization model.

- Human-in-the-loop profile editing workflows.
- Closed-loop ranking feedback and learning-to-rank.

5. Actors and User Roles

- Data Engineer: runs and monitors pipeline stages.
- ML/Backend Engineer: tunes ranking parameters and deploys API.
- Product/Analyst Consumer: consumes ranked researcher output.
- Downstream AI Agent: uses knowledge base and rank response.

6. Functional Context

The system receives topic definitions and query text, ingests scholarly data, produces a consolidated knowledge base, and exposes a ranking endpoint for retrieval.

7. User Journeys

Journey A: Build or refresh knowledge base

1. Operator runs pipeline (`python pipeline.py` with options).
2. Stage 1 discovers papers per concept/topic.
3. Stage 2 extracts and aggregates authors across concepts.
4. Stage 3 enriches profile/coauthor/institution data.
5. Stage 4 assembles researcher profiles and persists to DB.
6. Optional artifacts are exported (`knowledge_base.json`, summary Excel, schema JSON).

Success criteria:

- Pipeline completes without fatal errors.
- Data files and DB upserts reflect processed researchers.

Journey B: Rank researchers for a query

1. API client calls `POST /rank` with `query` (and optional filters/knobs).
2. Service computes Q (semantic relevance) and H (impact).
3. Service combines Q and H into final score and returns ranked results.
4. Response includes reasons and contribution summaries for explainability.

Success criteria:

- Valid response with sorted results.
- Clear debug metadata and stable error behavior.

8. Functional Requirements

FR-1 Topic-Based Discovery

The system shall discover papers from OpenAlex using configured topic/concept IDs and optional filters (region, date range, search query).

FR-2 Author Extraction and Consolidation

The system shall extract authors from discovered papers and consolidate per-author statistics across concepts.

FR-3 Profile Enrichment

The system shall enrich each author with profile and metrics from OpenAlex and, where available, Semantic Scholar.

FR-4 Coauthor and Mentorship Signals

The system shall infer collaboration and mentee-related signals from recent coauthor patterns.

FR-5 Institution Enrichment

The system shall enrich institution-level quality and geo metadata for current affiliations.

FR-6 Knowledge Base Assembly

The system shall build a normalized per-researcher profile object including identity, metrics, affiliations, topics, field relevance, paper summaries, and semantic scholar fields.

FR-7 Persistence

The system shall upsert institution/researcher/paper/topic/collaboration records to MySQL in batches with transaction handling and record-level savepoints.

FR-8 Artifact Export

The system shall export structured output artifacts:

- Knowledge base JSON
- Excel summary
- Summary column schema JSON

FR-9 Ranking Endpoint

The system shall expose `POST /rank` and accept:

- required: `query`
- optional: `region`, `institution_id`, `limit`, pareto flag, ranking knobs
- optional mock mode controls

FR-10 Health Endpoint

The system shall expose `GET /health` for liveness checks.

FR-11 Explainable Ranking Output

The rank response shall include:

- H, Q, final score
- reason summary/highlights
- contribution summary from matched papers
- debug metadata

FR-12 Ranking Modes

The system shall support two ranking modes:

- Simple mode (default): simplified H and Q
- Advanced mode: weighted multi-component H and decay/recency/citation-adjusted Q

9. Key Business Rules

- Query text must be non-empty.
- Ranking response defaults to top 25 if no `limit` is passed.
- Simple ranking mode is default unless explicitly overridden.
- Region/institution filters are applied to candidates before final scoring output.
- Stage 4 processes DB writes in batches and handles per-record failures with savepoints.

10. Functional Inputs and Outputs

Inputs

- Configured topic/concept list
- Runtime options (`--stage`, `--force`, `--limit`, filter/date args)
- API request body for ranking
- Environment variables for DB/Pinecone/embedding providers

Outputs

- Stage outputs in `data/raw`, `data/intermediate`, and final artifacts

- MySQL upserts of structured entities
- API JSON payload with ranked researchers and diagnostics

11. Error Handling (Functional Behavior)

- API translates known dependency/config errors to `503` and invalid request/value issues to `400`.
- Unexpected API/runtime errors return `500`.
- Stage 4 uses rollback + savepoint logic to skip bad records while continuing batch processing.

12. Assumptions and Constraints

- External data sources (OpenAlex, Semantic Scholar, Pinecone) are reachable.
- Embedding provider is configured and available for live ranking mode.
- Data quality depends on upstream source consistency.

13. Acceptance Criteria (Current-state)

1. Pipeline can run end-to-end and produce final artifacts.
2. Stage 4 writes records to MySQL with batch commit behavior.
3. `GET /health` returns status `ok`.
4. `POST /rank` returns sorted researcher results with explainability fields.
5. Mock ranking mode works without live vector/DB dependencies.

14. Future Functional Enhancements

- Human feedback loop on ranking quality.
- Stronger profile conflict resolution and dedup rules.
- Scheduled incremental refresh policies and SLA dashboards.
- Multi-query benchmarking and quality regression suite.